

《基于 Qt 的记忆配对游戏——千花万恋的设计与实现》

——C++ / Qt Widgets 课程设计项目报告

摘要

随着计算机图形界面技术的发展，传统的命令程序已经难以满足用户对于交互性、可视化以及娱乐性的需求。基于图形用户界面的小游戏由于其开发成本适中、功能实现完整、能够较好地锻炼程序设计能力，逐渐成为程序设计课程实践的重要方向。翻牌配对类游戏由于规则简单、逻辑清晰、具备一定的趣味性与可扩展性，非常适合作为 Qt 框架综合实践项目。

本人在本次课程设计中，基于 C++ 语言和 Qt6 Widgets 框架开发了一款记忆配对小游戏——《千花万恋》。该项目以经典翻牌配对玩法为核心，在实现基础逻辑功能的同时，对界面视觉效果、交互动画、多媒体播放以及系统设置模块进行了较为深入的优化设计。整个系统采用模块化设计思想，将主窗口管理、游戏逻辑、卡牌控件、动画按钮、系统设置以及匹配提示等功能拆分为独立模块，提高了程序整体的可维护性与可扩展性。

在界面实现方面，本项目采用 QStackedWidget 构建多页面切换结构，实现大厅界面、难度选择界面、游戏界面以及设置界面的统一管理；在交互设计方面，通过 QPropertyAnimation、QGraphicsDropShadowEffect 等 Qt 动画组件实现按钮呼吸光效、卡牌翻转动画以及匹配成功特效；在系统功能方面，利用 Qt Multimedia 模块实现背景音乐循环播放、翻牌音效、匹配音效以及胜利音效，并通过全局配置类 GameSettings 对音乐音量、音效开关以及翻牌速度进行统一管理，使设置界面能够直接影响实际游戏运行效果。

在开发过程中，本人还针对视频背景遮挡 UI、困难模式卡牌布局重叠、鼠标悬停导致控件偏移、胜利返回后游戏状态未重置以及动画速度参数失效等多个实际问题进行了调试与优化，最终形成了一个功能较为完整、界面表现良好、交互体验流畅的桌面小游戏。

关键词：Qt6；C++；Widgets；翻牌配对；动画设计；游戏开发

第一章 绪论

1.1 课题研究背景

在软件开发过程中，图形化程序的开发能力是程序员必须掌握的重要能力之一。Qt 作为跨平台 C++ 图形界面开发框架，具有组件丰富、信号槽机制灵活、动画系统完善以及多媒体支持方便等优点，被广泛应用于桌面软件开发、工业控制软件开发以及游戏辅助工具开发等领域。

传统的课堂案例往往集中在计算器、学生管理系统等功能型程序，其界面交互较为简单，对于 Qt 中布局管理、动画系统、多媒体播放、事件处理以及复杂状态控制等内容覆盖有限。因此，本人选择以小游戏开发作为课程设计方向，希望通过完整项目实践，将 Qt 各个模块进行综合运用。

翻牌配对游戏虽然玩法规则简单，但内部实现包含：

动态界面布局；

状态管理；

动画播放；

音效控制；

界面切换；

用户设置；

胜负判断；

这些功能覆盖了图形程序开发的大部分核心内容，因此具有较高的实践价值。

1.2 项目目标

本项目主要实现以下目标：

- （1）完成一款可独立运行的翻牌配对小游戏；
- （2）设计完整的大厅界面与模式选择界面；
- （3）实现简单模式与困难模式双难度；
- （4）实现卡牌翻转动画；

- (5) 实现卡牌匹配判定机制;
- (6) 加入背景音乐与交互音效;
- (7) 设计设置界面, 实现实时参数控制;
- (8) 优化整体 UI 视觉效果;
- (9) 提升游戏反馈感与完成度。

1.3 开发环境

本项目开发环境如下:

项目	环境
操作系统	Windows 11
编译器	MinGW64
开发语言	C++17
开发框架	Qt 6
构建工具	CMake
IDE	Qt Creator / VSCode

第二章 系统需求分析

2.1 功能需求分析

本系统主要包含四大模块:

(1) 大厅模块

提供:

开始游戏

游戏设置

退出游戏

本人将大厅设计为游戏入口界面，通过呼吸光效按钮增强视觉表现。

（2）模式选择模块

提供：

简单模式（5 对卡牌）

困难模式（8 对卡牌）

返回大厅

不同模式下：

卡牌数量不同；

布局不同；

卡牌尺寸不同。

（3）游戏模块

核心功能：

点击翻牌；

两张牌比较；

成功匹配；

匹配失败翻回；

全部完成触发胜利。

这是整个项目最核心的逻辑模块。

（4）设置模块

支持：

背景音乐音量调节；

游戏音效开关；

翻牌速度调整。

2.2 非功能需求

主要包括：

交互流畅性

动画不能卡顿。

可维护性

代码结构需要清晰。

美观性

界面需要具有游戏感。

第三章 系统总体设计

3.1 总体结构设计

项目主要类结构如下：

MainWindow

└— SettingsDialog

└— GameWindow(简单)

└— GameWindow(困难)

└— AnimatedButton

└— CardWidget

└─ MatchLabel
└─ GameSettings

3.2 主窗口设计

主窗口采用：

```
stack->addWidget(hallPage);  
stack->addWidget(modePage);  
stack->addWidget(settingPage);  
stack->addWidget(easyGame);  
stack->addWidget(hardGame);
```

利用 `QStackedWidget` 统一管理页面。

优点：

页面切换方便；

状态管理清晰；

不需要频繁 `new/delete` 窗口。

3.3 全局配置类设计

本人单独设计：

GameSettings

主要保存：

```
int musicVolume = 50;  
bool soundEnabled = true;  
int flipSpeed = 180;
```

统一共享配置。

例如：

```
GameSettings::flipSpeed
```

可以直接影响翻牌动画速度。

第四章 详细设计与实现

4.1 卡牌生成逻辑

在 `GameWindow::initCards()` 中：

```
for(int i=1;i<=pairCount;i++)
{
    ids.push_back(i);
    ids.push_back(i);
}
```

生成成对数字。

随后：

```
std::shuffle(
    ids.begin(),
    ids.end(),
    *QRandomGenerator::global()
);
```

随机打乱。

这样保证每局排列不同。

4.2 卡牌布局设计

简单模式：

```
cols=5;
cardSize=QSize(160,220);
```

困难模式：

```
cols=4;
cardSize=QSize(115,160);
```

使用：

```
grid->addWidget(card,i/cols,i%cols);
```

进行网格布局。

这种方法布局稳定。

4.3 点击逻辑设计

核心代码：

```
if(!firstCard)
{
    firstCard=card;
    return;
}
```

第一次点击记录。

第二次：

```
secondCard=card;
```

随后比较：

```
if(firstCard->cardId()==secondCard->cardId())
```

完成匹配判断。

这是典型双状态机设计。

4.4 翻牌动画实现

本人采用：

```
QPropertyAnimation
```

控制 geometry。

核心过程：

收缩：

```
QRect thinRect(...)
```

更新图案：

```
updateIcon();
```

展开恢复。

形成翻牌效果。

同时：

```
GameSettings::flipSpeed
```

控制速度。

设置页实时生效。

4.5 按钮动画设计

在 `AnimatedButton` 中：

加入：

```
QGraphicsDropShadowEffect
```

形成发光阴影。

再通过：

```
QPropertyAnimation
```

周期变化 `blurRadius`。

形成呼吸效果。

4.6 音效系统实现

播放接口：

```
GameSettings::playSound("flip.wav");
```

匹配：

```
playSound("match.wav");
```

胜利：

```
playSound("win.wav");
```

统一管理。

4.7 设置模块实现

音量：

```
GameSettings::musicVolume=value;
```

音效：

```
GameSettings::soundEnabled=checked;
```

翻牌速度：

```
GameSettings::flipSpeed=180;
```

设置完成后立即影响游戏。

4.8 胜利弹窗设计

本人重写 QMessageBox 样式：

```
msg.setStyleSheet(...)
```

设置：

深色背景；

金色按钮；

大字号标题；

增强完成感。

第五章 开发中的问题与优化

5.1 视频背景遮挡 UI

最初使用视频背景。

问题：

视频控件层级覆盖页面。

导致按钮不可见。

最终改为静态背景图解决。

5.2 困难模式重叠

原因：

卡牌过大。

布局空间不足。

解决：

动态区分尺寸。

5.3 卡牌偏移

原因：

Hover 时使用 move。

多次进入离开累计偏移。

解决：

统一恢复位置。

5.4 返回大厅不重置

增加：

```
resetGame();
```

返回时重置状态。

5.5 动画速度失效

原因：

动画拆分后总时长翻倍。

修改：

```
duration = flipSpeed/2;
```

恢复正常。

第六章 总结与心得

通过本次《千花万恋》项目开发，本人对于 Qt 图形程序开发有了更加系统的理解。从最初实现基础按钮，到逐步加入页面管理、布局设计、动画系统、多媒体控制以及全局配置管理，整个开发过程不仅提升了编程能力，也让我对软件工程中的模块化设计思想有了更深理解。

尤其是在问题调试过程中，我深刻认识到，一个完整的软件开发过程并不是单纯完成功能实现，更重要的是不断发现问题、分析问题并优化问题。从 UI 层级遮

挡，到布局冲突，再到状态重置与参数控制失效，每一个问题的解决都让我更加熟悉 Qt 框架的运行机制。

总体而言，《千花万恋》已经实现了课程设计预期目标，具备完整游戏流程、良好视觉表现以及较好的交互体验。同时，本项目仍具有进一步扩展空间，例如增加计时系统、排行榜系统、更多主题卡牌以及存档系统等。后续本人也希望在现有基础上继续完善，使其成为一个更加完整的独立小游戏作品。

参考文献

- [1] 霍亚飞. Qt 6 C++ 开发指南[M]. 北京：电子工业出版社.
- [2] 夏普. C++ Primer（第 5 版）[M]. 人民邮电出版社.
- [3] [Qt Documentation](#)
- [4] 孙鑫. C++ 面向对象程序设计教程[M]. 清华大学出版社.

附录：

AI 参考部分包括：

一、CardWidget.h

CardWidget.cpp

其中牌面翻转动画参考了 GPT，如：

```
shrink
```

```
→ expand
```

```
→ bounce
```

二、GameWindow.h

GameWindow.cpp

其中翻牌逻辑部分参考了 GPT，如：

```
if(!firstCard)
```

```
{
```

```
    firstCard = card;

    return;
}

secondCard = card;

busy = true;
```